Aleatoriedade vs Pseudoaleatoriedade:

Demonstrando a importância do gerador de chaves em Criptoprafia Simétrica através de uma simulação simplificada em *Python*

Gisela Ceresér Kassick & Dr. Daniel Roberto Cassar

IV Encontro do Programa de Pós-Graduação em Matemática IGCE-UNESP-Rio Claro

gisela25026@ilum.cnpem.br

Introdução

A segurança de sistemas criptográficos depende da produção de sequências imprevisíveis; portanto, a qualidade da fonte aleatória é determinante para a confidencialidade e a integridade das comunicações (GENNARO, 2006).

Formalmente, uma sequência aleatória deve apresentar três propriedades principais (ZHANG et al., 2021):

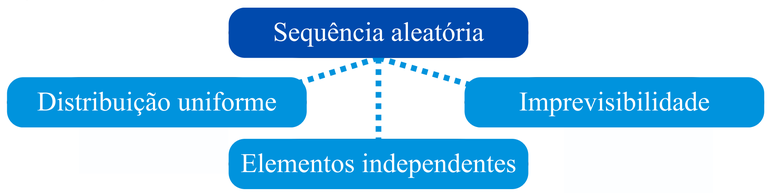

Figura 1.
FONTE: da autora

As fontes de aleatoriedade dividem-se em dois grupos. True Random Number Generators (TRGNs) utilizam fenômenos físicos imprevisíveis, como o ruído térmico, mas podem exigir técnicas de extração de entropia para reduzir vieses (CUCU LAURENCIU; COTOFANA, 2018; GENNARO, 2006). Já Pseudo Random Number Generators (PRNGs) geram sequências a partir da expansão determinística de uma semente; quando não projetados para criptografia, seu estado interno pode ser reconstruído (VIGNA, 2019). Em contraste, CSPRNGs são desenvolvidos para manter a imprevisibilidade mesmo sob análise parcial (D'APRANO, 2015).

Neste contexto, este trabalho investiga se diferenças na origem da aleatoriedade — física ou algorítmica — produzem variações estatísticas capazes de impactar a segurança de um sistema simples de criptografia simétrica.

Metodologia

O estudo adotou abordagem experimental comparativa, com 10.000 execuções por fonte e sequências de 256 bits, integrando Arduino e Python via comunicação serial.

- **Fontes avaliadas:**
  - 'random' (PRNG não criptográfico)
  - 'secrets' (CSPRNG do sistema operacional)
  - Fonte física bruta (ruído térmico via Arduino, mostrado na Figura 1)
  - Fonte física com extração por SHA-256
- **Análises realizadas:**
  - Frequência de 1s

$$f_1 = \frac{\sum_{i=1}^{N} b_i}{N}$$

  - Entropia de Shannon

$$H(X) = -\sum_i P(x_i)\log_2 P(x_i)$$

  - Autocorrelação

$$R(k) = \frac{E[(X_i-\mu)(X_{i+k}-\mu)]}{\sigma^2}$$

  - Teste qui-quadrado (uniformidade)

$$\chi^2 = \sum_{i=1}^{k} \frac{(O_i - E_i)^2}{E_i}$$

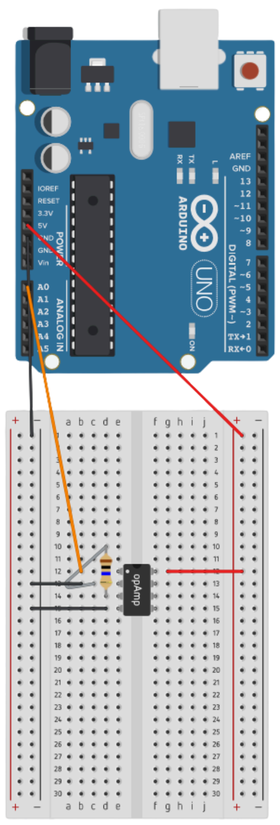

Figura 2.
FONTE: da autora, feito com *Tinkercad*

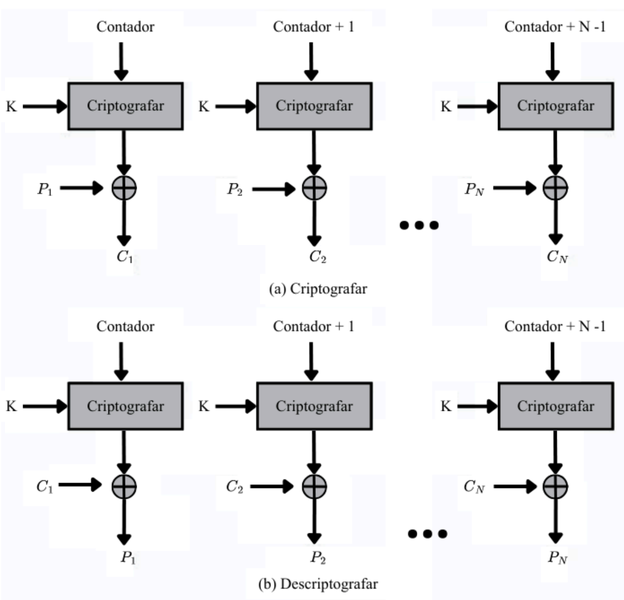

Figura 3.
FONTE: adaptado de BLAZHEVSKI et al (2012)

Por fim, os bits gerados derivaram a chave AES-256 e IV de 128 bits para cifragem com Advanced Encryption Standard (AES) em modo CTR. A segurança prática foi avaliada por uma série de ataques: por descoberta de seed, força bruta limitada por viés estatístico e ataque de força bruta limitado. Os resultados estatísticos e o desempenho sob ataque foram comparados para avaliar o impacto da origem da aleatoriedade.

Resultados e Discussão

	random()	secrets()	físico cru	físico + hash
frequência de 1s	0,499	0,5	0,466	0,5
entropia	0,997	0,997	0,976	0,997
autocorrelação	-0,004	-0,005	0,191	-0,004
p-valor	0,504	0,501	0,213	0,505

Tabela 1
FONTE: da autora

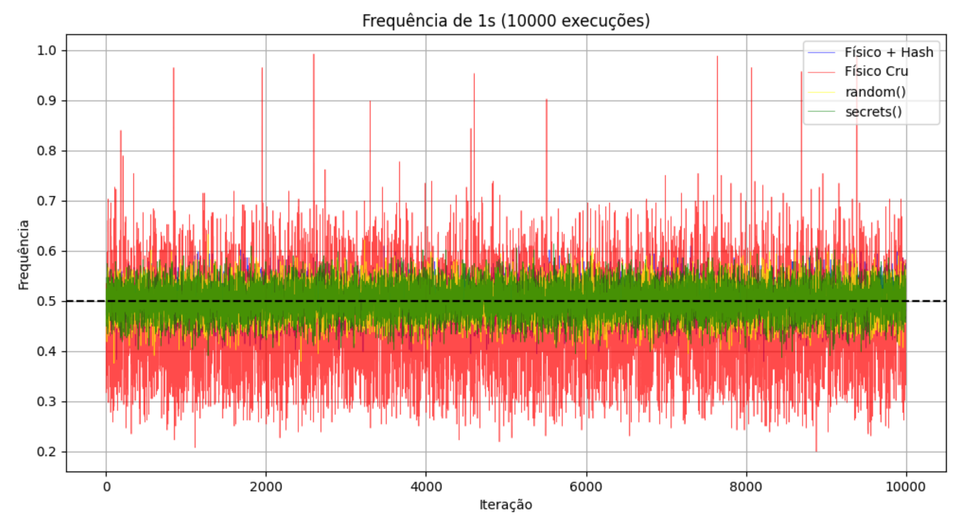

Figura 4.
FONTE: da autora

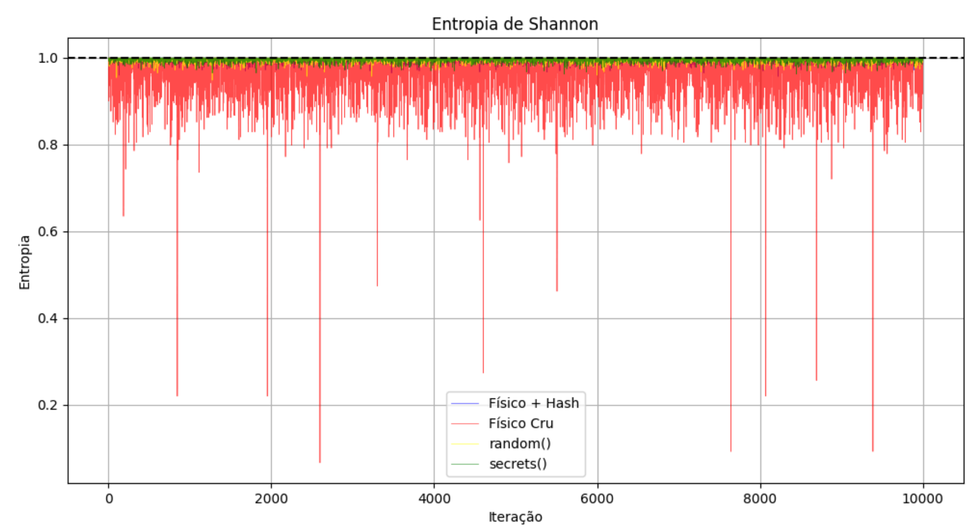

Figura 5.
FONTE: da autora

Considerações Finais

Em síntese, os resultados obtidos reforçam um princípio fundamental da criptografia: boas propriedades estatísticas não são suficientes para garantir segurança criptográfica (MENEZES; VAN OORSCHOT; VANSTONE, 1996). O gerador random() apresentou métricas estatísticas praticamente ideais, mas permaneceu vulnerável a ataques devido à sua natureza determinística. Por outro lado, a fonte física bruta apresentou imperfeições experimentais, como viés e autocorrelação, que podem reduzir a entropia efetiva da sequência gerada, fenômeno comum em sistemas físicos reais de geração de entropia (CUCU; COTOFANA, 2018). A aplicação de funções hash criptográficas demonstrou ser um mecanismo eficaz para mitigar essas imperfeições e produzir sequências mais adequadas para aplicações de segurança (GENNARO, 2006). Dessa forma, os resultados evidenciam a importância de utilizar algoritmos e arquiteturas especificamente projetados para criptografia ao construir sistemas de geração de números aleatórios destinados a aplicações seguras (PYTHON SOFTWARE FOUNDATION, 2024; VIGNA, 2025).

Referências

BLAZHEVSKI, D. et al. Modes of Operation of the AES Algorithm. 2012.
CUCU LAURENCIU, N.; COTOFANA, S. D. Delft University of Technology, 2018.
D'APRANO, S. PEP 506 – Adding a Secrets Module to the Standard Library. Python Enhancement Proposals, 2015.
GENNARO, R. IEEE Security & Privacy, v. 4, n. 2, p. 64–67, 2006.
KATZ, J.; LINDELL, Y. Introduction to Modern Cryptography. 2007.
PYTHON SOFTWARE FOUNDATION. random — geração de números pseudoaleatórios. Python Documentation.
PYTHON SOFTWARE FOUNDATION. secrets — geração de números seguros para gerenciamento de dados sensíveis. Python Documentation.
PYTHON SOFTWARE FOUNDATION. hashlib — algoritmos de hash seguros. Python Documentation.
PYCA CRYPTOGRAPHY. Cryptography Documentation.
VIGNA, S. arXiv:1910.06437, 2019.
WALTER, D.; BÜLAU, A.; ZIMMERMANN, A. Sensors, v. 23, n. 3, 1107, 2023.
ZHANG, M. et al. Journal of Physics: Conference Series, v. 1861, n. 1, 012009, 2021.

Repositório do GitHub